MERN STACK

TRAINING TR-103 REPORT DAY 30

29 July 2026

Overview:

Day 30 of the MERN Stack training focused on developing the Signup API for the CareerConnect – Career Management System. We learned how to create a user registration API using Node.js, Express.js, and MongoDB. The Signup API receives user registration details, checks whether the email already exists, creates a new user if the email is unique, and stores the user information in MongoDB using the User model.

Learning Objectives:

- Understand the working of a Signup API.
- Learn how to handle user registration using Express.js.
- Receive registration details through a POST request.
- Check whether an email already exists.
- Create and save a new user document.
- Store user information in MongoDB.
- Handle successful and failed registration responses.
- Understand error handling using `try-catch`.

Project: CarrerConnect – Carrer Guidance and Management System

Signup API Development

• The Signup API is developed to allow new users to register in the CareerConnect career management system.
• The API is created using Express.js and connected with MongoDB through the User model.
• The Signup API handles user registration requests using the POST method.

API Endpoint:

```
POST /signup
```

• The user registration data is received from the frontend in JSON format.
• Before creating a new account, the system checks whether the email already exists in the database.
• The email verification is performed using:

```
User.findOne({email:req.body.email})
```

• If the email already exists, the system returns:

```
Email already exists
```

• If the email is not found, a new User object is created using the received registration data.
• The user information is stored in MongoDB using:

```
user.save()
```

• After successful registration, the server sends a success response containing a success status and an Account Created Successfully message.
• Error handling is implemented using try-catch blocks to manage server-side errors.

Working

1. The app.post("/signup") route creates the Signup API.
2. User registration details are received through the request body.
3. The system searches the database for an existing email.
4. If the email already exists, registration is stopped and an error message is returned.
5. If the email is unique, a new User document is created.
6. The user data is saved into MongoDB using user.save().
7. A successful response is sent back to the frontend.
8. If any error occurs, an appropriate error response is returned.

Conclusion:

By the end of Day 30, I gained a clear understanding of developing a Signup API using Node.js, Express.js, and MongoDB. I learned how to receive registration data through a POST request, check for existing users using `findOne()`, create new user documents, and store them in MongoDB using `save()`. I also learned how to handle duplicate emails and server errors, strengthening my understanding of backend API development, user registration, MongoDB operations, and error handling in MERN Stack applications.

```
app.post("/signup",async(req,res)=>{
    try{
        const exist=await User.findOne({email:req.body.email});

        if(exist)
            return res.json({success:false,message:"Email already exists"});

        const user=new User(req.body);
        await user.save();

        res.json({
            success:true,
            message:"Account Created Successfully"
        });

    }catch(err){
        res.json({success:false,message:err.message});
    }
});
```